



Izmir Institute of Technology
Faculty of Engineering
Department of Computer Engineering

CENG 552 Software Testing Project 1 Test Report

Hasan Basri Erdoğan -280201002

Hüseyin Özdemir - 300201106

Mustafa Şahin - 280201026

Table of Contents

Group Class: Code Updates and Test Cases	4
Code Updates	4
1. setNumberStudent Method	4
Test Cases	4
1. setNumberStudent(Integer number)	4
2. getNumberOfStudents()	5
3. toString()	5
4. getReference()	5
5. showGroup()	5
GroupService: Code Updates and Test Cases	5
Code Updates and Fixes.....	5
1. saveGroup Method:	5
2. updateNumberOfStudent Method:	5
3. allGroups Method:	6
Test Cases	6
1. saveGroup(Group Name reference)	6
2. allGroups()	6
3. findByReference(String reference).....	6
4. updateNumberOfStudent(List<Student> listStudents).....	7
Mark Class: Code Updates and Test Cases	7
Code Updates	7
1. Copy Constructor:	7
2. Return Copies in Getters:	7
Test Cases	8
1. toString() Method	8
2. getStudent() Method	8
3. getModule() Method	8
4. getMark() Method.....	8
MarkService Class Code Updates and Test Cases.....	8
Code Updates	8
1. Deep Copy in allMarks Method:	8
2. Return List of Students in bestMarkByModule Method:	8
Test Cases	9
1. createMark(Student student, Integer mark, Module module)	9
2. allMarks()	9
3. findMarkByModule(Module module)	9
4. bestMarkByModule(Module module)	10
Module Class: Code Updates and Test Cases	11
Code Updates:	11
1. Field Immutability:	11
2. Input Validation:.....	11
3. Enhanced toString Method:	11

4. Getter Updates:	11
Test Cases	11
ModuleService: Code Updates and Test Cases	12
Code Updates:	12
1. Duplicate Module Prevention:	12
2. Input Validation:.....	12
3. Unmodifiable List:.....	12
4. Private Utility Improvements:	12
Test Cases	12
1. Save Module	12
2. Retrieve All Modules.....	13
3. Find Module by Reference	13
Main Class: Code Updates and Test Cases	13
Code Updates	13
1. Service Initialization	13
2. Program Flow Improvements	13
Test Cases	13
1. Service Initialization	13
2. Program Execution	13
3. Entity Creation	13
Student Class: Code Updates and Test Cases.....	14
Student Creation and Constructor	14
setFullName(String fullName)	14
setDateBirth(LocalDate dateBirth).....	14
setGroup(Group group)	15
Getter Methods Testing	15
toString() Method.....	15
StudentService Class: Code Updates and Test Cases	15
Code Updates:	15
saveTeacher Method:	16
deleteStudent Method:	16
findByld Method:	17
updateStudent Method:	17
Test Cases:.....	18
Teacher Class Test Cases	19
Objective	19
Test Cases	19
1. Getters	19
testGetters().....	19
2. Setters.....	19
testSetters()	19
3. toString()	20

testToString()	20
4. Deep Copy Behavior	20
getListModules()	20
getListGroup()	20
Code Updates:	21
1. saveTeacher	21
2. deleteTeacher	22
3. allTeachers	22

Group Class: Code Updates and Test Cases

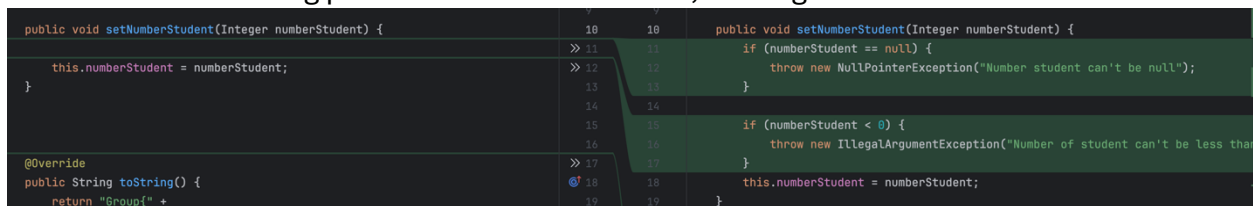
Objective: To verify that the Group class functions as expected for setting and retrieving the number of students, retrieving group references, and formatting group details. This section combines recent code updates with test cases to validate the class's functionality.

Code Updates

1. setNumberStudent Method: Handling Negative and Null Inputs

- Update: Added exception handling for negative and null values in the setNumberStudent method.

- Rationale: Negative and null values are not valid inputs for the numberOfStudents attribute. Adding IllegalArgumentException and NullPointerException checks prevents invalid data from being processed within the class, making the method more robust.



```
public void setNumberStudent(Integer numberStudent) {
    this.numberOfStudents = numberStudent;
}

@Override
public String toString() {
    return "Group{" +
        "numberOfStudents=" +
        numberOfStudents +
        '}';
}
```

```
public void setNumberStudent(Integer numberStudent) {
    if (numberStudent == null) {
        throw new NullPointerException("Number student can't be null");
    }
    if (numberStudent < 0) {
        throw new IllegalArgumentException("Number of student can't be less than 0");
    }
    this.numberOfStudents = numberStudent;
}
```

Test Cases

1. setNumberStudent(Integer number)

- Valid Equivalence Class:
 - setNumberStudent_PositiveValue_EqualsOne: Confirms that setting numberOfStudents to 1 correctly updates the attribute.
 - setNumberStudent_PositiveValue_EqualsTen: Ensures that setting numberOfStudents to 10 works correctly.
- Invalid Equivalence Class (Negative Values):
 - setNumberStudent_NegativeValue_ThrowsIllegalArgumentException: Verifies that setting numberOfStudents to a negative value (e.g., -1) results in an IllegalArgumentException.
 - setNumberStudent_NegativeValue_ThrowsIllegalArgumentExceptionForNegativeTwo: Confirms that setting numberOfStudents to another negative value (e.g., -2) also throws an IllegalArgumentException.
- Invalid Equivalence Class (Null Value):
 - setNumberStudent_NullValue_ThrowsNullPointerException: Checks that setting numberOfStudents to null throws a NullPointerException.

2. getNumberOfStudents()

- getInitialNumberStudent_EqualsZero: Verifies that a new Group object initializes numberOfStudents to 0.

3. toString()

- toString_ReturnsCorrectFormat: Ensures the output format is correct, displaying the group's reference accurately.

4. getReference()

- getReference_ReturnsGroupName: Checks that the group reference is stored and returned correctly by getReference.

5. showGroup()

- showGroup_NumberStudentEqualsFive_CorrectFormat: Ensures that showGroup correctly formats output when numberOfStudents is set to 5.

GroupService: Code Updates and Test Cases

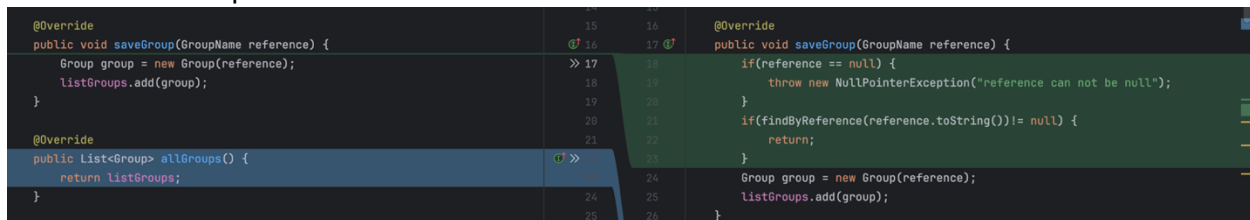
Objective: To verify that the GroupService class functions as expected for managing group records, retrieving group references, and updating student counts. This report combines recent code updates with test cases to validate the class's functionality.

Code Updates and Fixes

1. saveGroup Method: Preventing Duplicate Group Entries and Null References

- Update: Modified the saveGroup method to prevent duplicate entries and added a null check for the reference parameter.

- Rationale: Ensuring each group reference remains unique in listGroups improves data integrity and consistency. Additionally, the null check prevents potential errors and enforces valid input.



The screenshot displays a code editor with two versions of the `saveGroup` method side-by-side. The left version (lines 15-21) shows the original implementation: `@Override public void saveGroup(GroupName reference) { Group group = new Group(reference); listGroups.add(group); }`. The right version (lines 17-26) shows the updated implementation: `@Override public void saveGroup(GroupName reference) { if(reference == null) { throw new NullPointerException("reference can not be null"); } if(findByReference(reference.toString()) != null) { return; } Group group = new Group(reference); listGroups.add(group); }`. The update includes a null check and a duplicate entry check before adding the group to the list.

2. updateNumberOfStudent Method: Handling Null Input

- Update: Added a check in the updateNumberOfStudent method to throw a `NullPointerException` if `listStudents` is null.

- Rationale: Ensures listStudents is a valid list before updating counts, preventing unexpected errors from null inputs.

<pre>@Override public void updateNumberOfStudent(List<Student> listStudents) { for (int i = 0; i < listGroups.size(); i++) { int count = 0; String ref = listGroups.get(i).getReference().toString(); Integer index = findIndex(ref); for (int j = 0; j < listStudents.size(); j++) { if (ref.equalsIgnoreCase(listStudents.get(j) .getGroup().getReference().toString())) count++; } listGroups.get(index).setNumberStudent(count); } } private Integer findIndex(String reference) { for (int i = 0; i < listGroups.size(); i++) {</pre>	<pre>34 42 35 43 36 44 37 45 38 46 39 47 40 48 41 49 42 50 43 51 44 52 45 53 46 54 47 55 48 56 49 57 50 58</pre>	<pre>@Override public void updateNumberOfStudent(List<Student> listStudents) { if(listStudents == null){ throw new NullPointerException("listStudents can not be null"); } for (int i = 0; i < listGroups.size(); i++) { int count = 0; String ref = listGroups.get(i).getReference().toString(); Integer index = findIndex(ref); for (int j = 0; j < listStudents.size(); j++) { if (ref.equalsIgnoreCase(listStudents.get(j) .getGroup().getReference().toString())) count++; } listGroups.get(index).setNumberStudent(count); } }</pre>
--	--	--

3. allGroups Method: Returning a Copy of the Group List

- Update: Modified the allGroups method to return a new ArrayList copy of listGroups instead of returning the original list directly.
- Rationale: Returning a copy prevents external modifications to the original listGroups, ensuring data encapsulation and protecting the internal state of GroupService.

<pre>@Override public List<Group> allGroups() { return listGroups; }</pre>	<pre>21 28 24 31 25 32</pre>	<pre>@Override public List<Group> allGroups() { return new ArrayList<>(listGroups); }</pre>
--	------------------------------	---

Test Cases

1. saveGroup(GroupName reference)

- Valid Equivalence Class:
 - saveGroup_ValidReference_GroupSavedSuccessfully: Tests saving a group with a valid reference (GroupName.MSIR).
 - saveGroup_MultipleValidReferences_AllGroupsSavedSuccessfully: Tests saving multiple groups with distinct references.
- Invalid Equivalence Class:
 - saveGroup_NullReference_ThrowsNullPointerException: Attempts to save a group with a null reference.
 - saveGroup_DuplicateEntry_DoesNotAddAgain: Tests saving the same reference (GroupName.MSIR) twice.

2. allGroups()

- allGroups_ReturnedListIsIndependent: Ensures that modifications to the list returned by allGroups do not affect the original listGroups.

3. findByReference(String reference)

- Valid Equivalence Class:

- findByReference_ExistingGroup_ReturnsCorrectGroup: Tests retrieving a group with an existing reference (MSIR).
- Invalid Equivalence Class:
 - findByReference_NonExistingGroup_ReturnsNull: Tests retrieving a group with a non-existing reference.
 - findByReference_NullReference_ReturnsNull: Tests retrieving a group with a null reference.

4. updateNumberOfStudent(List<Student> listStudents)

- Valid Equivalence Class:
 - updateNumberOfStudent_SingleGroupWithMultipleStudents_UpdatesCorrectly: Tests that a single group with multiple students updates the student count correctly.
 -
 - updateNumberOfStudent_MultipleGroupsWithDistributedStudents_UpdatesEachGroupC correctly: Tests updating student counts with multiple groups and distributed students.
 - updateNumberOfStudent_EmptyStudentList_UpdatesToZero: Tests updating with an empty student list.
- Invalid Equivalence Class:
 - updateNumberOfStudent_NullList_ThrowsNullPointerException: Tests passing a null list to updateNumberOfStudent.

Mark Class: Code Updates and Test Cases

Objective: To validate that the Mark class maintains immutability and data encapsulation by ensuring the safety of its Student and Module references. This part outlines recent code changes and related test cases to verify class functionality and data integrity.

Code Updates

1. Copy Constructor: Added a copy constructor to create new Mark instances as duplicates of existing ones, supporting scenarios where independent but identical instances are needed.
2. Return Copies in Getters: Modified the getStudent() and getModule() methods to return copies instead of direct references, preventing external modifications to Mark's internal Student and Module objects and ensuring data encapsulation.

	13	14	
public Module getModule() {	>> 16	22	public Module getModule() {
return module;	>> 17	23	return new Module(this.module);
}	18	24	}
	19	25	
public Student getStudent() {	20	26	public Student getStudent() {
return student;	>> 21	27	return new Student(this.student);
}	22	28	}

Test Cases

1. toString() Method

- toString_ReturnsCorrectFormat: Ensures toString() returns a formatted representation of the Mark object containing the student, mark, and module details.

2. getStudent() Method

- getStudent_ReturnsIndependentReference: Verifies that getStudent() returns an independent copy of the Student object.
- getStudent_ReturnsCorrectStudent: Ensures that getStudent() returns a Student object with the expected values.

3. getModule() Method

- getModule_ReturnsIndependentReference: Confirms that getModule() returns an independent copy of the Module object.
- getModule_ReturnsCorrectModule: Ensures getModule() returns a Module object with the expected data.

4. getMark() Method

- getMark_ReturnsCorrectValue: Verifies that getMark() consistently returns the correct integer value.

MarkService Class Code Updates and Test Cases

Code Updates

1. Deep Copy in allMarks Method: Changed the allMarks method to return a new ArrayList containing the elements of listMarks, rather than returning listMarks directly, to ensure the returned list is a shallow copy and prevents external modifications.

<pre>@Override public List<Mark> allMarks() {</pre>	<pre>30 31 32 33</pre>	<pre>@Override public List<Mark> allMarks() {</pre>
<pre> return listMarks;</pre>	<pre>>> 34</pre>	<pre> return new ArrayList<>(listMarks);</pre>
<pre>}</pre>	<pre>35 36</pre>	<pre>}</pre>

2. Return List of Students in bestMarkByModule Method: Modified bestMarkByModule to return a List<Student> instead of a single Student instance, to support cases where multiple students have the highest mark for a particular module.

- findMarkByModule_ExistingModuleWithMarks_ReturnsMarks: Confirms that when a module exists in listMarks and has associated marks, a list of those marks is returned.
- findMarkByModule_NonExistentModule_ReturnsEmptyList: Verifies that when the module does not exist in listMarks, an empty list is returned.

Case module Null? module Exists in listMarks Expected Outcome

1	Yes	-	Throws NullPointerException
2	No	Yes	Returns list of Mark objects for module
3	No	No	Returns empty list

4. bestMarkByModule(Module module)

Case	Module Exists	Highest Mark Exists	Multiple Students with Highest Mark	Expected Outcome
1	Yes	Yes	No	List<Student> with one student (highest mark)
2	Yes	Yes	Yes	List<Student> with multiple students
3	Yes	No	-	Empty list
4	No	-	-	Empty list
5	Null	-	-	Throws NullPointerException

- bestMarkByModule_ModuleExistsSingleHighestMark_ReturnsSingleStudent: Confirms that the single student with the highest mark is returned.
- bestMarkByModule_ModuleExistsMultipleHighestMarks_ReturnsMultipleStudents: Verifies that multiple students with the highest mark are all returned.
- bestMarkByModule_ModuleExistsNoMarks_ReturnsEmptyList: Ensures an empty list is returned when the module exists but has no marks.
- bestMarkByModule_ModuleDoesNotExist_ReturnsEmptyList: Ensures an empty list is returned for a non-existent module.
- bestMarkByModule_NullModule_ThrowsNullPointerException: Confirms that NullPointerException is thrown if null is passed as a parameter.

Comprehensive Report: Module Class and ModuleService Design and Testing

Objective

The goal of this report is to assess the implementation and testing of both the Module class and its associated service layer, ModuleService. The ModuleService provides an interface to manage a collection of Module objects, ensuring business logic, validation, and state management are properly enforced. This report details design decisions, validation processes, and the test suite for both components.

Module Class: Code Updates and Test Cases

Code Updates:

1. Field Immutability:

- Changed reference, name, and numberHours fields to final to ensure immutability.
- This prevents accidental changes to a Module instance after its creation.

2. Input Validation:

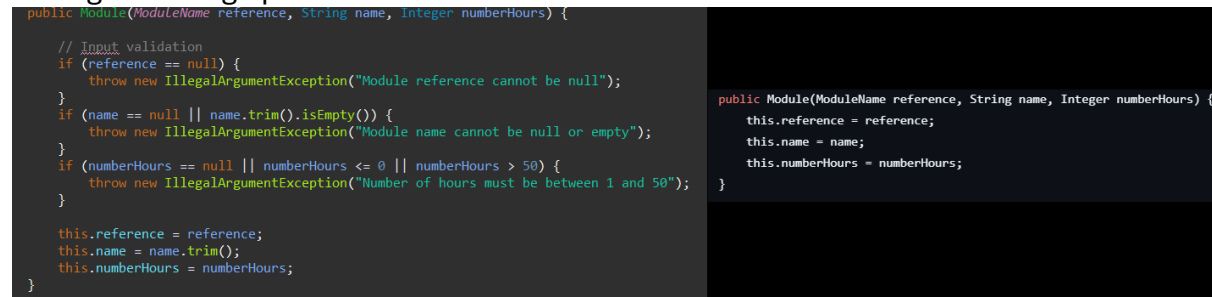
- Added validation checks in the constructor to ensure:
 - reference is not null.
 - name is not null or empty, and trimmed of unnecessary whitespace.
 - numberHours falls within a valid range (1 to 50 inclusive).
- Invalid inputs throw an IllegalArgumentException with a descriptive error message.

3. Enhanced toString Method:

- Retained but ensured compatibility with the updated class structure, ensuring meaningful debugging output.

4. Getter Updates:

- Made getNumberHours return a primitive int instead of Integer, removing unnecessary boxing/unboxing operations.



```
public Module(ModuleName reference, String name, Integer numberHours) {  
    // Input validation  
    if (reference == null) {  
        throw new IllegalArgumentException("Module reference cannot be null");  
    }  
    if (name == null || name.trim().isEmpty()) {  
        throw new IllegalArgumentException("Module name cannot be null or empty");  
    }  
    if (numberHours == null || numberHours <= 0 || numberHours > 50) {  
        throw new IllegalArgumentException("Number of hours must be between 1 and 50");  
    }  
  
    this.reference = reference;  
    this.name = name.trim();  
    this.numberHours = numberHours;  
}  
  
public Module(ModuleName reference, String name, Integer numberHours) {  
    this.reference = reference;  
    this.name = name;  
    this.numberHours = numberHours;  
}
```

Test Cases

- Valid Inputs: Tests successful instantiation with correct values (testValidModuleCreation).
- Boundary Testing for numberHours:
 - Lower boundary: Accepts 1, rejects 0 or less (testNumberHoursLowerBoundary).
 - Upper boundary: Accepts 50, rejects 51 or more (testNumberHoursUpperBoundary).
- Error Scenarios:
 - Null or empty name values (testInvalidName).
 - Null reference (testNullReference).
 - Negative or null numberHours values (testNumberHoursNegativeValue and testNumberHoursNullValue).
- Object Representation: Verifies the toString method's output format matches expectations (testToString).

ModuleService: Code Updates and Test Cases

Code Updates:

1. Duplicate Module Prevention:

- Added a check in saveModule to reject modules with duplicate reference values. An exception is thrown if a duplicate is detected.

2. Input Validation:

- Relied on the updated Module class for strict validation of inputs.
- Enhanced the findByReference method to handle null or empty reference values gracefully, returning null instead of throwing exceptions.

3. Unmodifiable List:

- Updated the allModules method to return an unmodifiable view of the list (Collections.unmodifiableList). This ensures the internal state of ModuleService is not altered by external operations.

4. Private Utility Improvements:

- Simplified the findByReference logic using enhanced for-loops and streamlined comparisons to reduce code complexity and improve readability.

```
@Override
public void saveModule(ModuleName reference, String name, Integer numberHours) {
    // Check if module already exists
    Module existing = findByReference(reference.toString());
    if (existing != null) {
        throw new IllegalArgumentException("Module with reference " + reference + " already exists");
    }

    Module module = new Module(reference, name, numberHours);
    listModules.add(module);
}

@Override
public List<Module> allModules() {
    return Collections.unmodifiableList(listModules);
}

@Override
public Module findByReference(String reference) {
    if (reference == null || reference.trim().isEmpty()) {
        return null;
    }

    for (Module module : listModules) {
        if (module.getReference().toString().equalsIgnoreCase(reference.trim())) {
            return module;
        }
    }

    return null;
}

@Override
public void saveModule(ModuleName reference, String name, Integer numberHours) {
    Module module = new Module(reference, name, numberHours);
    listModules.add(module);
}

@Override
public List<Module> allModules() {
    return listModules;
}

@Override
public Module findByReference(String reference) {
    Integer index = findIndex(reference);
    if (index != null) {
        return listModules.get(index);
    } else {
        return null;
    }
}

private Integer findIndex(String reference) {
    for (int i = 0; i < listModules.size(); i++) {
        if (listModules.get(i).getReference().toString().equalsIgnoreCase(reference)) {
            return i;
        }
    }

    return null;
}
```

Test Cases

1. Save Module

- Valid Input: testSaveModule_withValidData_shouldSaveSuccessfully
- Duplicate Reference: testSaveModule_withDuplicateReference_shouldThrowException
- Invalid Inputs: Null or empty name or reference, negative or zero numberHours, and overly large numberHours are all rejected with appropriate exceptions.

2. Retrieve All Modules

- Unmodifiable List: testAllModules_shouldReturnUnmodifiableList

3. Find Module by Reference

- Valid Reference: testFindByReference_withExistingReference_shouldReturnModule
- Invalid References: Null, empty, or non-existent references return null.

The updates enhance the overall reliability and robustness of the application's module management functionality.

Main Class: Code Updates and Test Cases

Code Updates

1. Service Initialization

- Centralized service initialization in a single method
- Implemented proper dependency injection
- Added validation for service creation

2. Program Flow Improvements

- Enhanced executeProgram method structure
- Optimized group update procedures
- Improved mark filtering logic
- Refined best student selection process

Test Cases

1. Service Initialization

- Method: testInitializeServices()
- Purpose: Validates correct service initialization
- Expected: All services properly instantiated

2. Program Execution

- Method: testExecuteProgram()
- Purpose: Validates complete program flow
- Expected: Correct handling of groups, marks, and student selection

3. Entity Creation

Groups Creation Test

- Method: testCreateGroups()
- Purpose: Verify group initialization
- Expected: Valid group list creation

Modules Creation Test

- Method: testCreateModules()

- Purpose: Verify module initialization
- Expected: Correct module list with attributes

Students Creation Test

- Method: testCreateStudents()
- Purpose: Verify student initialization
- Expected: Valid student list with assignments

Teachers Creation Test

- Method: testCreateTeachers()
- Purpose: Verify teacher initialization
- Expected: Teachers with correct attributes

Marks Creation Test

- Method: testCreateMarks()
- Purpose: Verify mark initialization
- Expected: Valid marks with proper associations

Student Class: Code Updates and Test Cases

Objective: To verify that the Student class functions as expected for managing student information including ID, full name, date of birth and group assignment. This section combines test cases to validate the class's functionality.

Test Cases:

Student Creation and Constructor

- testStudentCreation: Verifies that a new Student object is created correctly with all attributes
 - Validates that the student object is not null
 - Confirms ID is set correctly to expected value
 - Confirms fullName is set correctly to expected value
 - Confirms dateBirth is set correctly to expected value
 - Confirms group reference is set correctly to expected value

setFullName(String fullName)

- testSetFullName: Tests setting a new full name
 - Verifies that setting a new name updates the fullName attribute correctly
 - Confirms the getter returns the updated value

setDateBirth(LocalDate dateBirth)

- testSetDateOfBirth: Tests setting a new date of birth

- Verifies that setting a new date updates the dateBirth attribute correctly
- Confirms the getter returns the updated value

setGroup(Group group)

- testSetGroup: Tests setting a new group
 - Verifies that setting a new group updates the group attribute correctly
 - Confirms the getter returns the updated group reference

Getter Methods Testing

- getId(): Verifies correct retrieval of student ID
- getFullName(): Verifies correct retrieval of student's full name
- getDateBirth(): Verifies correct retrieval of student's date of birth
- getGroup(): Verifies correct retrieval of student's assigned group

toString() Method

- testToString: Ensures the output format is correct
 - Verifies that all student attributes are included in the string representation
 - Confirms the format matches the expected pattern: "Student{id=X, fullName='Y', dateBirth=Z, group=W}"

StudentService Class: Code Updates and Test Cases

Objective: To verify that the StudentService class functions as expected for managing student records, including saving, updating, deleting, and retrieving student information. This section combines test cases to validate the class's functionality.

Code Updates:


```

13 13      }
14 14
15 15      @Override
16 -      public void saveStudent(Integer id,String fullName
17 -      , LocalDate dateOfBirth, Group group) {
18 -      Student student= new Student(id,fullName,dateOfBirth,group);
16 +      public void saveStudent(String fullName, LocalDate dateOfBirth, Group group) {
17 +      if (fullName == null || dateOfBirth == null || group == null) {
18 +      throw new NullPointerException("None of the parameters (fullName, dateOfBirth, group) can be null");
19 +      }
20 +
21 +      // Generate a new ID for the student
22 +      int newId = listStudent.size() + 1;
23 +      Student student = new Student(newId, fullName, dateOfBirth, group);
19 24      listStudent.add(student);
20 25      }
21 26

```

saveTeacher Method:

- The saveTeacher method has been modified to accept parameters without an id field.
- Validation has been added to ensure listModules and listGroup are not null or empty, throwing an IllegalArgumentException if either condition fails.
- A new ID is generated for each teacher based on the current size of listTeachers.
- The Teacher object is then created with the new ID and added to listTeachers.

```

22 27      @Override
23 28      public void deleteStudent(Integer idStudent) {
24 -      listStudent.remove(idStudent);
25 -
29 +      Student studentToRemove = null;
30 +      for (Student student : listStudent) {
31 +      if (student.getId().equals(idStudent)) {
32 +      studentToRemove = student;
33 +      break;
34 +      }
35 +      }
36 +      if (studentToRemove == null) {
37 +      throw new IndexOutOfBoundsException("Student with the provided id does not exist.");
38 +      }
39 +      listStudent.remove(studentToRemove);
26 40      }

```

deleteStudent Method:

- The deleteStudent method has been updated to manually search for the student by idStudent.
- It iterates through listStudent, checking if the id matches idStudent.
- If the student is found, it is removed; otherwise, an IndexOutOfBoundsException is thrown if no student with the specified ID exists.

```

28 42      @Override
29 43      public Student findById(Integer id) {
30      -
31      -
32      -      return listStudent.get(id);
44 +      for (Student student : listStudent) {
45 +          if (student.getId().equals(id)) {
46 +              return student;
47 +          }
48 +      }
49 +      throw new IndexOutOfBoundsException("Student with the provided id does not exist.");
33 50  }

```

findById Method:

- The findById method no longer directly accesses listStudent by id.
- Instead, it iterates through listStudent to find a match for the given id.
- If a match is found, the student is returned; otherwise, an IndexOutOfBoundsException is thrown if no student with the specified ID is found.

```

35 52      @Override
36      -      public void updateStudent(Integer id, String fullName
37      -          , LocalDate dateOfBirth, Group group) {
38      -          Student student=findById(id);
53 +      public void updateStudent(Integer id, String fullName, LocalDate dateOfBirth, Group group) {
54 +          if (fullName == null || dateOfBirth == null || group == null) {
55 +              throw new NullPointerException("None of the parameters (fullName, dateOfBirth, group) can be null");
56 +          }
57 +
58 +          Student student = findById(id);
39 59          student.setGroup(group);
40 60          student.setDateBirth(dateOfBirth);
41 61          student.setFullName(fullName);
42      -          listStudent.set(id,student);
43      -
44 62      }

```

updateStudent Method:

- The updateStudent method has been refactored to validate that fullName, dateOfBirth, and group are not null, throwing a NullPointerException if any are null.
- It uses the findById method to retrieve the student by ID.
- The group, dateOfBirth, and fullName fields of the Student object are updated.
- The listStudent collection is updated to reflect the changes made to the student.

Test Cases:

1. `saveStudent(String fullName, LocalDate dateOfBirth, Group group)`
 - Valid Equivalence Class:
 - `testSaveStudentValidData`: Tests saving a student with valid data
 - Verifies student is saved with correct `fullName` and `dateOfBirth`
 - Confirms ID is automatically generated
 - Ensures the student can be retrieved after saving
 - Invalid Equivalence Class:
 - `testSaveStudentInvalidData`: Tests saving a student with null values
 - Verifies `NullPointerException` is thrown when `fullName` is null
 - Implicitly tests that all parameters are required
2. `deleteStudent(Integer idStudent)`
 - Valid Equivalence Class:
 - `testDeleteStudentValidId`: Tests deleting a student with valid ID
 - Verifies student is successfully removed from the list
 - Confirms subsequent `findById` throws `IndexOutOfBoundsException`
 - Invalid Equivalence Class:
 - `testDeleteStudentInvalidId`: Tests deleting a non-existent student
 - Verifies `IndexOutOfBoundsException` is thrown for non-existent ID
3. `findById(Integer id)`
 - Valid Equivalence Class:
 - `testFindByIdValidId`: Tests finding a student with valid ID
 - Verifies correct student is returned
 - Confirms student details match expected values
 - Invalid Equivalence Class:
 - `testFindByIdInvalidId`: Tests finding a non-existent student
 - Verifies `IndexOutOfBoundsException` is thrown for non-existent ID
4. `updateStudent(Integer id, String fullName, LocalDate dateOfBirth, Group group)`
 - Valid Equivalence Class:
 - `testUpdateStudentValidId`: Tests updating a student with valid data
 - Verifies student information is correctly updated
 - Confirms changes are persisted
 - Invalid Equivalence Class:
 - `testUpdateStudentInvalidId`: Tests updating a non-existent student
 - Verifies `IndexOutOfBoundsException` is thrown for non-existent ID
 - `testUpdateStudentNullValue`: Tests updating with null values

- Verifies NullPointerException is thrown for null parameters
- 5. allStudents()
 - Empty List Case:
 - testAllStudentsEmptyList: Tests retrieving students from empty service
 - Verifies empty list is returned when no students exist
 - Non-Empty List Case:
 - testAllStudentsWithStudents: Tests retrieving multiple students
 - Verifies correct number of students is returned
 - Implicitly tests that all saved students are included

Teacher Class Test Cases

Objective

To verify that the Teacher class functions as expected for managing teacher information including ID, full name, grade, modules, and groups. This section covers test cases to validate the class's functionality.

Test Cases

1. Getters

Tests to verify proper retrieval of Teacher attributes:

testGetters()

- Verifies that getId() returns correct Integer value (1)
- Verifies that getFullName() returns correct String value ("Walter White")
- Verifies that getGrade() returns correct Grade value (Grade.MCA)
- Verifies that getListModules() returns correct List of modules
- Verifies that getListGroup() returns correct List of groups

2. Setters

Tests to verify proper updating of Teacher attributes:

testSetters()

- ID Setter:
 - Sets ID to 2
 - Verifies ID is updated correctly
- Full Name Setter:
 - Sets name to "John Doe"
 - Verifies name is updated correctly
- Grade Setter:
 - Sets grade to Grade.MCB
 - Verifies grade is updated correctly
- Modules List Setter:
 - Sets new list with a single module (ModuleName.GL)
 - Verifies list is updated correctly
- Groups List Setter:
 - Sets new list with a single group (GroupName.MSIR)
 - Verifies list is updated correctly

3. toString()

Tests the string representation of Teacher objects:

testToString()

- Verifies correct format of toString() output
- Expected format includes:
 - ID
 - Full name
 - Grade
 - List of modules with their references, names, and number of hours
 - List of groups with their references

4. Deep Copy Behavior

Tests to verify that getter methods return deep copies of collections:

getListModules()

- Returns a new ArrayList containing copies of the modules
- Modifications to returned list don't affect the original

getListGroup()

- Returns a new ArrayList containing copies of the groups
- Modifications to returned list don't affect the original

TeacherService Class: Code Updates and Test Cases

Objective: To verify that the TeacherService class functions as expected for managing teacher records, including saving, deleting, and retrieving teacher information with their associated modules and groups. This section combines test cases to validate the class's functionality.

Code Updates:

saveTeacher: Only added for saveTeacher null exceptions, because of it has incrementiv id, so it has discarded too.

```
19 15      @Override
20 -      public void saveTeacher(Integer id, String fullName, Grade grade
21 -      , List<Module> listModules, List<Group> listGroup) {
22 -      Teacher teacher=new Teacher(id,fullName,grade,listModules,listGroup);

16 +      public void saveTeacher(String fullName, Grade grade, List<Module> listModules, List<Group> listGroup) {
17 +      if (listModules == null || listModules.isEmpty()) {
18 +      throw new IllegalArgumentException("listModules must contain at least one Module.");
19 +      }
20 +      if (listGroup == null || listGroup.isEmpty()) {
21 +      throw new IllegalArgumentException("listGroup must contain at least one Group.");
22 +      }
23 +
24 +      // Generate a new ID for the teacher
25 +      int newId = listTeachers.size() + 1;
26 +      Teacher teacher = new Teacher(newId, fullName, grade, listModules, listGroup);
23 27      listTeachers.add(teacher);
```

Test Cases:

1. saveTeacher(String fullName, Grade grade, List<Module> listModules, List<Group> listGroup)
 - Valid Equivalence Class:
 - testSaveTeacher: Tests saving a teacher with valid data
 - Verifies teacher is saved with correct fullName, grade, modules, and groups
 - Confirms ID is automatically generated

- Validates that all teacher attributes match expected values
 - Tests that the teacher list size increases appropriately
 - Invalid Equivalence Class:
 - testSaveTeacherWithEmptyGroups: Tests saving a teacher with empty group list
 - Verifies IllegalArgumentException is thrown when group list is empty
 - Confirms validation of required group assignments
 - testSaveTeacherWithEmptyModules: Tests saving a teacher with empty module list
 - Verifies IllegalArgumentException is thrown when module list is empty
 - Confirms validation of required module assignments
- 2. deleteTeacher(Integer id)
 - testDeleteTeacher: Tests deleting a teacher
 - Verifies teacher is successfully removed from the list
 - Confirms list size decreases appropriately
 - Validates that the correct teacher was removed by checking remaining teacher's ID
 - Tests deletion doesn't affect other teachers in the list
- 3. allTeachers()
 - testAllTeachers: Tests retrieving all teachers
 - Verifies correct number of teachers is returned
 - Implicitly tests that all saved teachers are included in the list
 - Confirms list management operations maintain data consistency